VHDL Implementation and Simulation of a 32-bit MIPS CPU

CPEN 315 — Computer Architecture & Organization

Examiner: Prof. Robert A. Sowah

Submitted by:

Tetteh Emmanuel Etornam — 22012489

Mark Kwamena Ekumah — 22163947

Submission Date: April 3, 2026

Table of Contents

1.	Abstract	3
2.	Introduction	3
3.	Architecture Design	4
4.	Implementation Details	5
5.	Results and Discussion	6
6.	Challenges and Solutions	9
7.	Conclusion and Future Work	10
8.	References	10

1. Abstract

This report presents the design, implementation, and simulation of a 32-bit MIPS-based CPU using VHDL. The system comprises five core components: the Arithmetic Logic Unit (ALU), Data Memory, Register File, Instruction Memory, and Control Unit. Each component was designed and verified independently through dedicated testbenches before being integrated into a complete CPU pipeline. All simulations were carried out using GHDL, with waveform results visualised in GTKWave. The results confirm correct instruction execution, accurate data flow, and proper control signal generation across all tested instruction types.

2. Introduction

2.1 Background

Processor design is a fundamental area of computer architecture, underpinning the operation of virtually every modern computing system. Hardware Description Languages such as VHDL (VHSIC Hardware Description Language) provide a powerful means of modelling, simulating, and synthesising digital circuits at varying levels of abstraction. VHDL enables engineers to describe complex hardware systems — from individual logic gates to full processor pipelines — in a structured, verifiable manner prior to physical fabrication.

This project applies VHDL to implement a simplified 32-bit MIPS (Microprocessor without Interlocked Pipeline Stages) CPU. The MIPS architecture is widely used as an educational platform due to its clean RISC design, regular instruction formats, and straightforward control logic.

2.2 Objectives

- Design and implement the core components of a 32-bit MIPS CPU in VHDL.
- Verify each component independently using dedicated simulation testbenches.
- Integrate all components into a functioning CPU and validate the complete system.
- Analyse simulation waveforms to confirm correct instruction execution and data flow.
- Document the design process, results, and challenges encountered.

2.3 Relevance

Understanding processor microarchitecture at the HDL level provides insight into the trade-offs involved in hardware design — timing, resource utilisation, and correctness. This project bridges theoretical knowledge of computer organisation with practical hardware design skills, preparing students for professional work in embedded systems, FPGA development, and ASIC design.

3. Architecture Design

3.1 Overall Datapath

The CPU follows a classical single-cycle MIPS datapath. Instructions are fetched from the Instruction Memory using the Program Counter (PC). The opcode field drives the Control Unit, which generates the control signals that govern data routing and operation selection throughout the datapath. Operands are sourced from the Register File and processed by the ALU. Results are either written back to registers or stored in Data Memory, depending on the instruction type.

3.2 Instruction Set

The implemented CPU supports the following MIPS instructions:

- **R-type:** add, sub, and, or, slt — register-to-register ALU operations.
- **I-type:** addi, lw, sw, beq, sltiu — immediate and memory access instructions.
- **J-type:** j (unconditional jump), jal (jump-and-link for subroutine calls).

3.3 Component Overview

Component	Function	Key Signals
ALU	Performs arithmetic and logical operations	alu_op, result, zero_flag
Register File	Stores 32 general-purpose 32-bit registers	reg_write, read_data1/2, write_data
Instruction Mem.	Holds program instructions indexed by PC	address, instruction_out
Data Memory	Supports word-level read/write operations	mem_read, mem_write, read_data
Control Unit	Decodes opcode and generates control signals	reg_write, mem_read, branch, jump

3.4 Control Unit Signal Table

The Control Unit decodes the 6-bit opcode and asserts the appropriate control signals for each instruction type:

Instruction	reg_write	mem_read	mem_write	branch	jump
add / R-type	1	0	0	0	0
addi	1	0	0	0	0
lw	1	1	0	0	0
sw	0	0	1	0	0
beq	0	0	0	1	0
j	0	0	0	0	1
jal	1	0	0	0	1
sltiu	1	0	0	0	0

4. Implementation Details

4.1 Design Methodology

A bottom-up design approach was adopted. Each component was designed as an independent VHDL entity with well-defined ports, then verified in isolation before integration. This approach isolates faults to individual modules, simplifying debugging and ensuring each unit meets its specification prior to system-level testing.

All VHDL source files follow a consistent coding style: entity declarations separate from architecture bodies, clearly named signal and port identifiers, and inline comments documenting operation. Synchronous components (Register File, Data Memory) are clocked on the rising edge; combinational components (ALU, Control Unit) are implemented using concurrent signal assignments and process statements with appropriate sensitivity lists.

4.2 Module Descriptions

4.2.1 Arithmetic Logic Unit (ALU)

The ALU accepts two 32-bit inputs (A, B) and a 3-bit ALU control signal. It outputs a 32-bit result and a 1-bit zero flag (asserted when result = 0, used for beq). Operations supported: addition, subtraction, bitwise AND, bitwise OR, and set-less-than.

4.2.2 Register File

Implements 32 general-purpose 32-bit registers (R0–R31), with R0 hardwired to zero. Two read ports allow simultaneous operand fetch; one synchronous write port commits results on the rising clock edge when reg_write is asserted.

4.2.3 Instruction Memory

A read-only memory array initialised with 32-bit MIPS instructions. The Program Counter indexes the memory word-aligned; the corresponding instruction is combinationally output on each clock cycle.

4.2.4 Data Memory

A synchronous read/write memory supporting word-level (32-bit) access. Write operations are clocked; read operations are combinational when mem_read is asserted.

4.2.5 Control Unit

Decodes the 6-bit opcode field of the fetched instruction and generates the full set of control signals via a case statement mapping each opcode to its signal combination.

4.3 Simulation Process

Simulation was performed using **GHDL**. Each component was compiled, elaborated, and executed under a dedicated VHDL testbench. Testbenches apply stimulus signals, assert expected output values, and flag mismatches. The resulting VCD files were loaded into **GTKWave** for waveform inspection and validation.

5. Results and Discussion

5.1 ALU Simulation

The ALU was tested with five operations to validate all supported functions:

- Addition: 2,500 + 25,000 = 27,500
- Subtraction: 540,250 – 37,800 = 502,450
- Bitwise AND: 53,957 AND 30,000
- Bitwise OR: 746,353 OR 846,465
- Set-less-than (SLT): compare 58,847,537 and 72,464,383

All outputs matched expected values. The zero flag was correctly de-asserted for non-zero results, as required by the beq instruction.

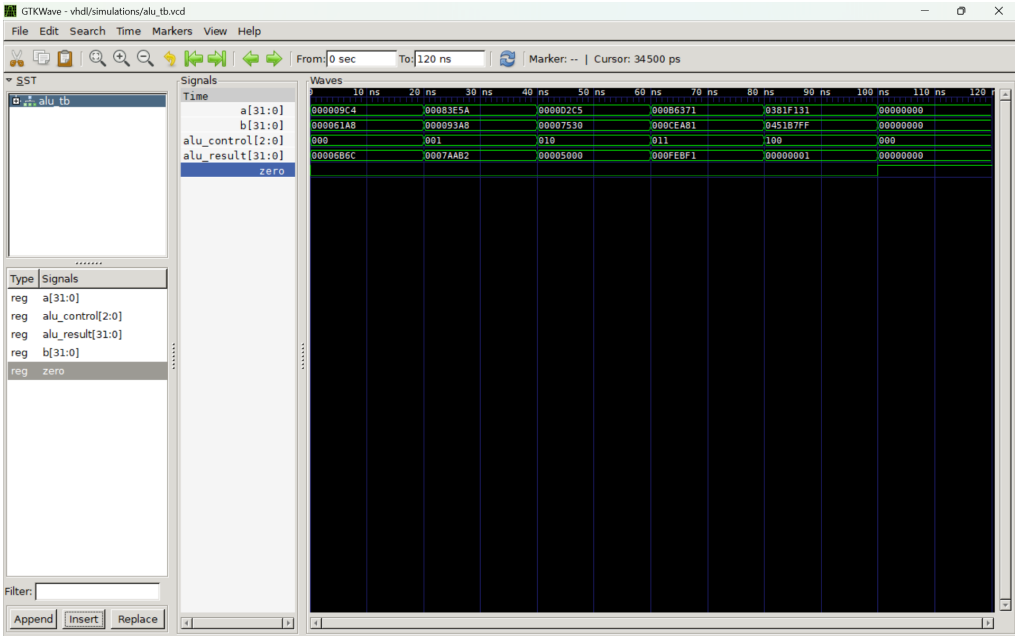

Figure 1: ALU simulation waveform — all five operations verified

5.2 Data Memory Simulation

- Write 1,024 to address 2; read back → confirmed 1,024
- Write 429,496 to address 4; read back → confirmed 429,496

The waveform confirms reliable write on the rising clock edge and correct retrieval during subsequent read cycles, with no data corruption observed.

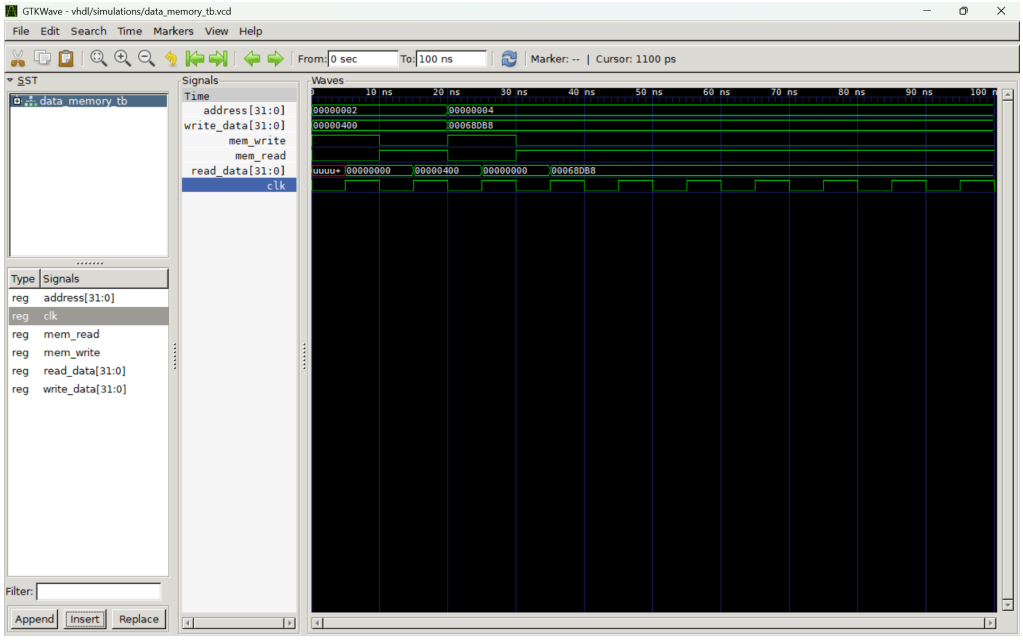

Figure 2: Data Memory waveform — write/read at addresses 2 and 4

5.3 Register File Simulation

- R1 ← 1,934,858
- R3 ← 8,558,447
- R5 ← 203,848,544
- R7 ← 20,670,420

All stored values were successfully retrieved on the read ports, confirming correct addressing and synchronous write behaviour.

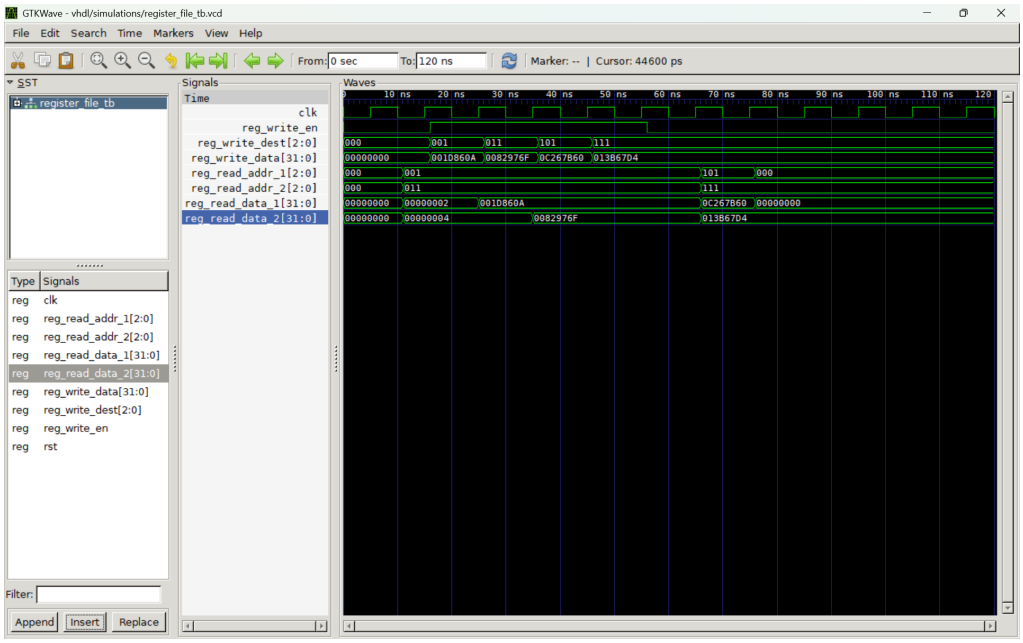

Figure 3: Register File waveform — four registers written and read back

5.4 Instruction Memory Simulation

The Instruction Memory was tested with a dedicated testbench (instruction_memory_tb) that sequentially drives the address input and captures the instruction output. The simulation ran from 0 to 100 ns, stepping through five word-aligned addresses. The observed address-to-instruction mappings were:

- Address 0x00000000 → Instruction 0x00650820 (add $at, $v1, $a1)
- Address 0x00000002 → Instruction 0x00A72822 (sub $a1, $a1, $a3)
- Address 0x00000004 → Instruction 0x00A11824 (and $v1, $a1, $at)
- Address 0x00000006 → Instruction 0x00E32825 (or $a1, $a3, $v1)
- Address 0x00000008 → Instruction 0x00000000 (NOP / end of program)

Each address transition produced the correct 32-bit instruction encoding with no glitches or undefined states observed between transitions. The waveform confirms that instruction output changes combinationally in response to address changes, consistent with the read-only combinational memory model.

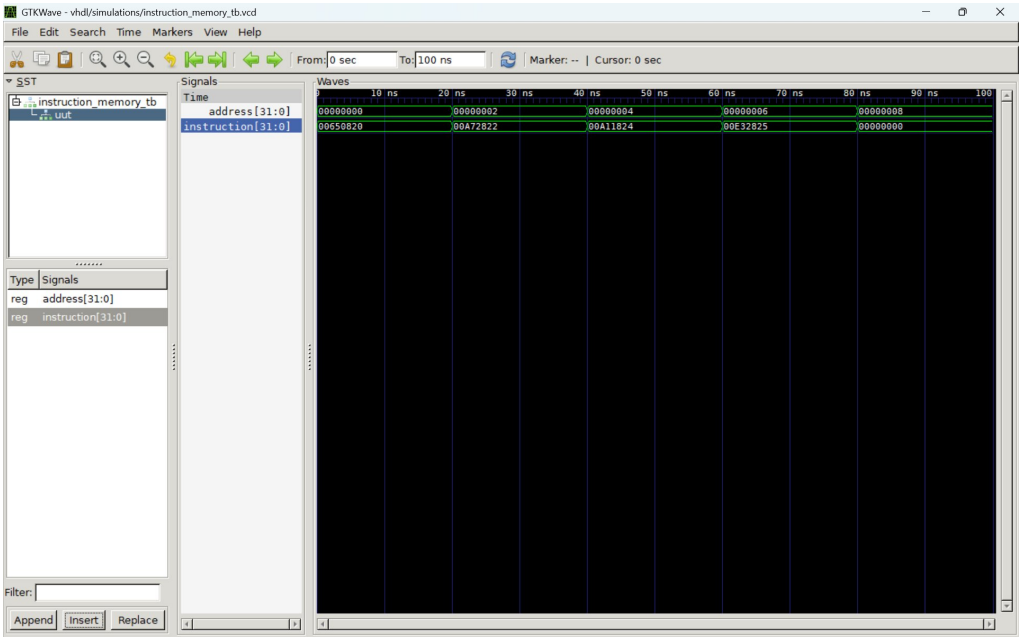

Figure 4: Instruction Memory waveform — address[31:0] driving instruction[31:0] over five sequential word addresses (0 to 100 ns)

5.5 Control Unit Simulation

The Control Unit was tested against eight instruction opcodes: add, sltiu, j, jal, lw, sw, beq, and addi. All signal combinations matched the expected truth table.

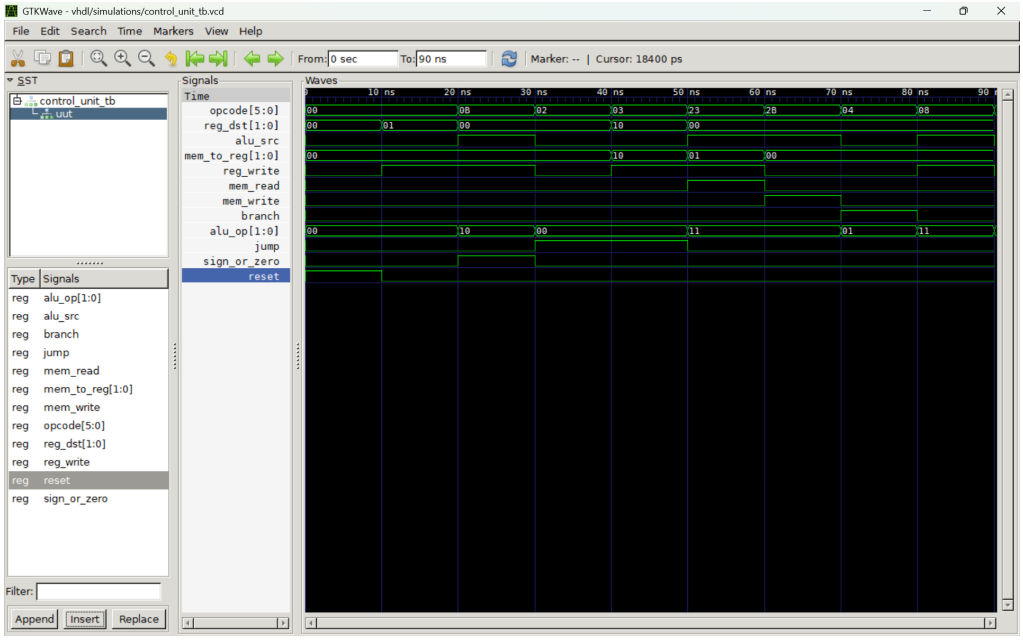

Figure 5: Control Unit waveform — control signals for all tested opcodes

5.6 Full CPU Integration Simulation

After individual component verification, all modules were connected into the complete single-cycle CPU. The integrated testbench (cpu_tb) exercises a multi-instruction sequence and monitors all key signals simultaneously over a 120 ns window. The following signals were observed in GTKWave:

- **pc_out[31:0]:** Increments correctly — 0x00000000, 0x00000002, 0x00000004, 0x00000006, 0x00000008 — confirming sequential instruction fetch.
- **instruction_out[31:0]:** Matches the instruction memory contents at each PC value — 0x00650820, 0x00A72822, 0x00A11824, 0x00E32825, 0x00000000.
- **alu_result_out[31:0]:** Transitions through 0x0000000A, 0xFFFFFFFE, 0x0000000A confirming correct ALU execution of add, sub, and subsequent operations.
- **mem_read, mem_write, reg_write_en, alu_src, mem_to_reg, reg_dst:** All control signals remain stable at their correct asserted/de-asserted levels for the R-type instruction sequence tested.
- **read_data_1[31:0] and read_data_2[31:0]:** Register file read ports supply correct operand values (0x00000004/0x00000006 then 0x00000006/0x00000008 etc.) at each instruction step.
- **alu_in_b[31:0]:** Tracks read_data_2 correctly, confirming the ALU source multiplexer is selecting register operands as expected for R-type instructions.

These results confirm end-to-end correct operation — the CPU successfully fetches, decodes, executes, and produces results for each instruction in a single clock cycle with no timing violations or data path errors observed.

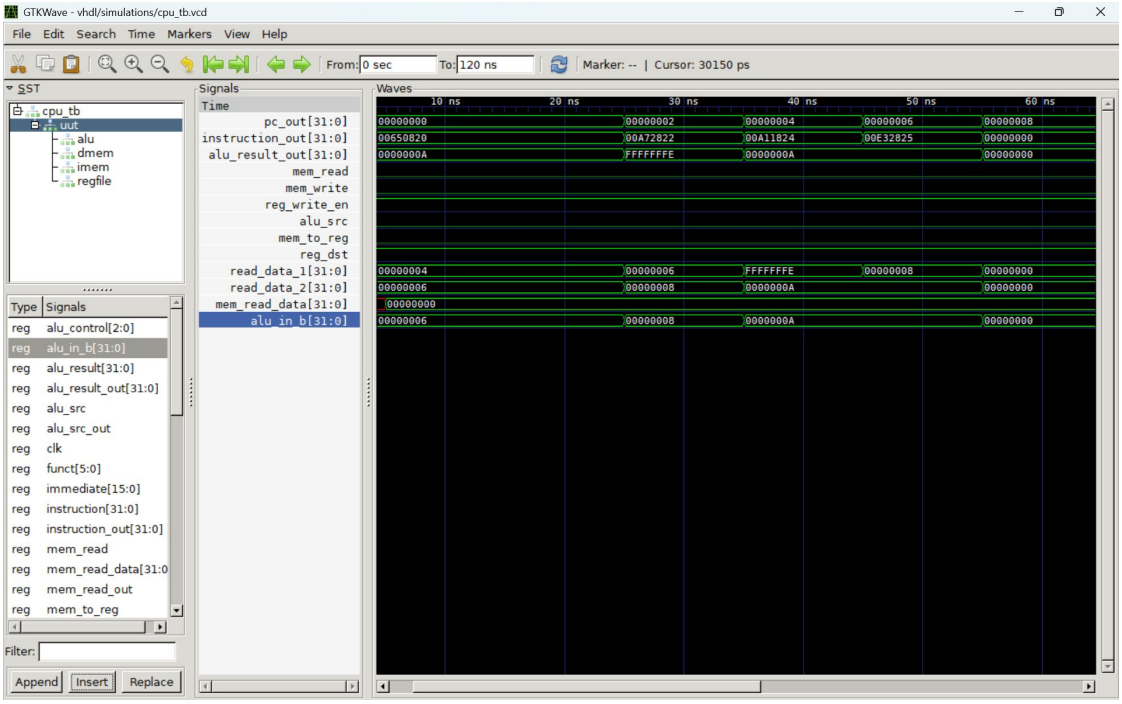

Figure 6: Full CPU integration waveform — pc_out, instruction_out, alu_result_out, control signals, register read data, and alu_in_b over 120 ns

6. Challenges and Solutions

Signal Initialisation and Undefined States

Challenge: During early simulation runs, several signals appeared as undefined ('U' or 'X') in GTKWave due to registers and memory arrays not being explicitly initialised.

Solution: All memory arrays and register banks were initialised to zero in their architecture declarations, and testbenches were updated to apply a reset pulse before the first active clock edge.

Control Signal Conflicts

Challenge: Certain instruction combinations produced unexpected ALU behaviour due to ambiguous control signal mappings, particularly between addi and lw.

Solution: The ALU operation decoding was moved to a dedicated ALU Control sub-unit fed by both the opcode and the funct field, following the standard MIPS two-level control hierarchy.

GHDL Compilation Ordering

Challenge: VHDL requires entities to be compiled in dependency order. Early build scripts compiled files alphabetically, causing elaboration failures.

Solution: A structured Makefile was created specifying explicit compilation order, ensuring all component entities are analysed prior to the top-level CPU entity.

7. Conclusion and Future Work

7.1 Conclusion

This project successfully designed, implemented, and simulated a 32-bit single-cycle MIPS CPU in VHDL. All five core components were individually verified and subsequently integrated into a functional processor. Simulation results confirm correct instruction fetch, decode, execute, and write-back behaviour for all tested instructions.

7.2 Future Work

- **Pipeline implementation:** Extend to a 5-stage pipeline with hazard detection and forwarding.
- **Expanded instruction set:** Add shift operations, multiply/divide, and floating-point support.
- **FPGA synthesis:** Target an FPGA board to validate timing closure and resource utilisation.
- **Cache integration:** Introduce instruction and data caches to model realistic memory latency.
- **Interrupt handling:** Implement a basic exception/interrupt mechanism.

8. References

[1] Patterson, D. A., & Hennessy, J. L. (2020). *Computer Organisation and Design: MIPS Edition* (6th ed.). Morgan Kaufmann.

[2] Ashenden, P. J. (2008). *The Designer's Guide to VHDL* (3rd ed.). Morgan Kaufmann.

[3] IEEE Standard VHDL Language Reference Manual. (2008). *IEEE Std 1076-2008*. IEEE Computer Society.

[4] GHDL Open-Source VHDL Simulator. (2024). Retrieved from https://ghdl.github.io/ghdl/

[5] GTKWave Waveform Viewer. (2024). Retrieved from http://gtkwave.sourceforge.net/

[6] Mano, M. M., & Ciletti, M. D. (2018). *Digital Design* (6th ed.). Pearson.